Online Order Processing with the Facade Pattern

Assignment 4 | Software Design Patterns

Adilzhan Kuandykov | SE-2526
Course ShP-2216 | OP 6B06102 | 18 October 2026

1 Introduction

The project implements Option B, an online order processing system in Java 17. OrderFacade provides two operations, placeOrder(cart, customer) and cancelOrder(orderId), over four services: inventory, payment, shipping, and notification. This report explains the class relationships, coordination logic, and Clean Code decisions in the implementation.

A purchase requires several dependent steps. Stock must be available before payment is taken, and a shipment should be registered only after payment succeeds. If payment is declined, reserved stock must become available again. Without a facade, every client would need to know the service APIs, the order of calls, and the cleanup rules. The facade centralizes that workflow so the console client can express a business request with one call.

1 1 Processing and cancellation

During placement, InventoryService validates the complete cart, calculates the total, and reserves stock. PaymentProcessor debits the customer balance, ShippingService records the delivery address, and NotificationService stores the confirmation. Only then does the facade save and return an Order with status PLACED.

If a later placement step fails, the facade cancels the shipment, refunds any recorded payment, and releases the reservation. Each cleanup operation safely does nothing when there is no matching entry. Inventory validation finishes before any stock is changed, so a rejected cart cannot leave a partial reservation.

Cancellation reverses the shipment, payment, and stock reservation, stores a CANCELLED order, and records a notification. A second cancellation returns that stored order without another refund or stock increase. An unknown order ID is rejected.

1 2 Scope

The services perform real changes to in-memory maps and lists. Payments, delivery, and messages are simulations; no external transaction or email is sent. Prices and balances use whole cents. The application uses only the Java standard library and is intended for a single thread. All data is reset when the process ends.

2 UML class diagram

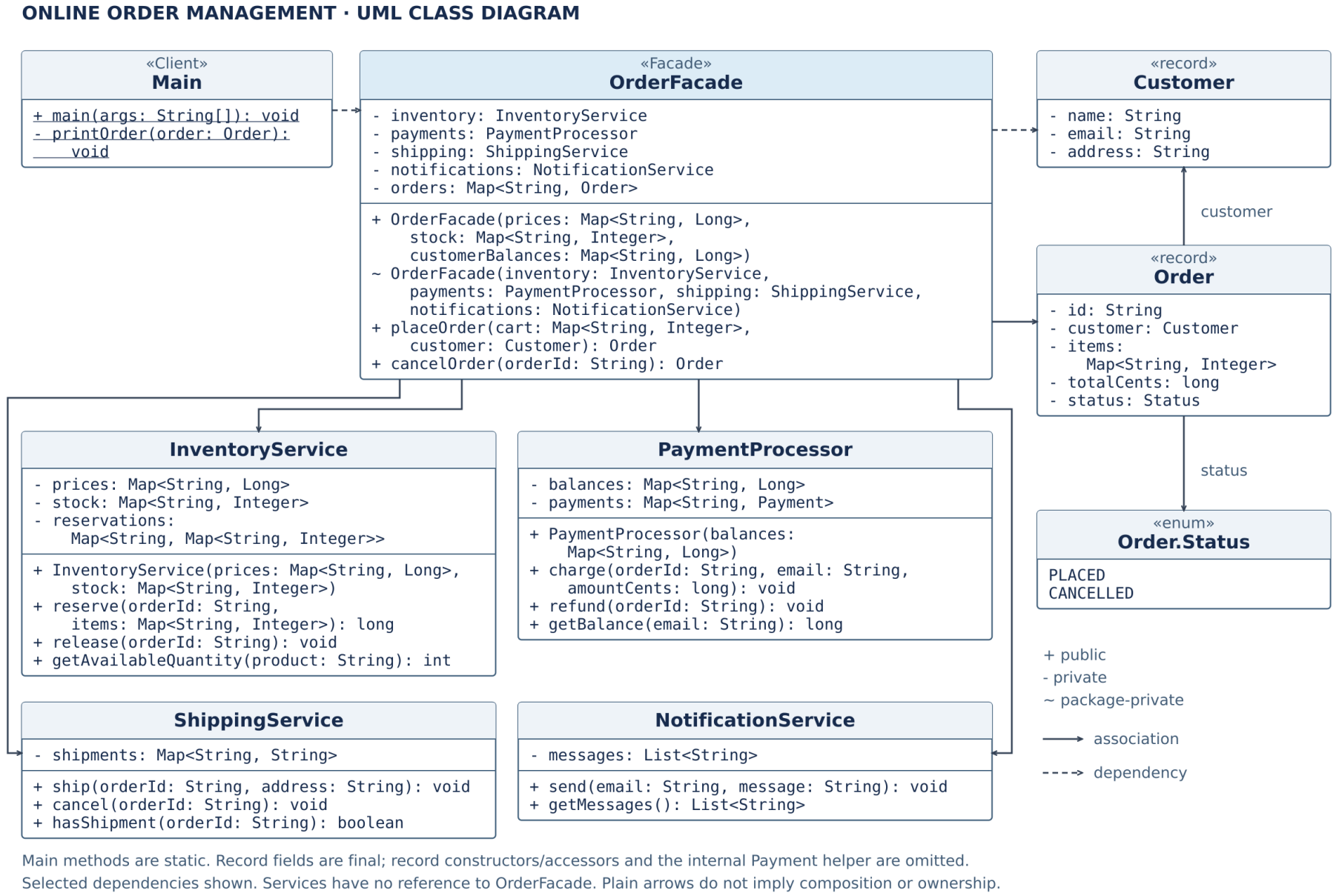


Figure 1. Classes and dependencies in the implemented system. Main uses the facade and data records; no subsystem depends on OrderFacade. Service links are associations because the package-private constructor can receive existing service instances.

3 Clean Code principles

Each example below is an excerpt from the committed Java source. The explanation beneath it annotates the relevant design decision without adding comments to the program.

3 1 Meaningful names

```
  public Order placeOrder(Map<String, Integer> cart, Customer customer) {
```

Source: OrderFacade.java, line 34.

placeOrder states the business action, and cart and customer identify its inputs. The returned Order is a domain result. The companion method cancelOrder(orderId) is equally explicit. These names let the client describe an order workflow without knowing how stock, payments, or shipment records are stored.

3 2 Single Responsibility in the facade

```
  payments.charge(orderId, customer.email(), totalCents);
  shipping.ship(orderId, customer.address());
  notifications.send(customer.email(), "Order " + orderId + " placed");
```

Source: OrderFacade.java, lines 45-47.

The facade selects the order of operations and passes the same order ID between services. PaymentProcessor performs the balance update; ShippingService manages shipment records; NotificationService stores the message. The facade does not duplicate those operations. Likewise, InventoryService calculates the total before these calls, keeping pricing and stock rules together.

3 3 A public API without subsystem details

```
  public OrderFacade(Map<String, Long> prices, Map<String, Integer> stock,
                     Map<String, Long> customerBalances) {
      this(new InventoryService(prices, stock), new PaymentProcessor(customerBalances),
              new ShippingService(), new NotificationService());
  }
```

Source: OrderFacade.java, lines 20-24.

The public constructor accepts catalog prices, stock quantities, and customer balances as ordinary data. It creates the required services internally. The public business methods accept a cart, a customer, or an order ID and return Order. There are no public service getters. The separate constructor that accepts service objects has package visibility and supports tests without exposing those dependencies to the normal client.

3 Clean Code principles continued

3 4 Small and focused subsystem classes

```
public void cancel(String orderId) {
    shipments.remove(orderId);
}

public boolean hasShipment(String orderId) {
    return shipments.containsKey(orderId);
}
```

Source: service/ShippingService.java, lines 19-25.

ShippingService owns a map from order IDs to delivery addresses. Its operations register, cancel, and check shipments; it has no payment or inventory logic. The other services follow the same boundary: InventoryService owns reservations, PaymentProcessor owns balances and payments, and NotificationService owns stored messages. A change to one responsibility therefore has a clear location.

3 5 Explicit error handling and compensation

```
} catch (RuntimeException failure) {
    shipping.cancel(orderId);
    payments.refund(orderId);
    inventory.release(orderId);
    throw new IllegalStateException("Could not place order: " + failure.getMessage(), failure);
```

Source: OrderFacade.java, lines 51-55.

The catch block compensates in reverse order: shipment, payment, then inventory. A payment rejection releases the reservation, while a shipping failure also refunds the payment. The new exception retains the original cause, which helps diagnose the failed step. Cleanup methods remove their recorded entry before restoring resources, so calling them repeatedly does not duplicate a refund or stock release.

3 6 Defensive copying of order data

```
public record Order(String id, Customer customer, Map<String, Integer> items,
                    long totalCents, Status status) {
    public Order {
        items = Map.copyOf(items);
    }
```

Source: model/Order.java, lines 5-9.

Order is a record, and its constructor copies the item map. Changing the caller's cart later cannot change the stored order, and callers cannot edit order.items(). This matters during cancellation: the quantities restored must match the original reservation. The facade and inventory service also take their own snapshots before processing mutable input.

3 7 Limits of the error handling

Compensation assumes that the in-memory cleanup methods complete normally. The example does not provide database transactions, retries, or protection against concurrent updates. Cancellation stores its final state before sending the notification; it does not roll back a completed cancellation if a replacement notification service throws. These limits keep the example focused on the Facade pattern.

4 Validation and demonstration

The project was compiled with JDK 17 using --release 17, -Xlint:all, and -Werror. The dependency-free test runner completed all 12 checks successfully. Shipping and notification failures are simulated by test subclasses; the tests then inspect the actual service state to verify compensation.

Checks	Count	Verified result
Placement and public API	2	Correct totals, state changes, and facade construction
Payment, shipping and notification failures	3	Stock, balances, and shipments are restored
Reservation and input validation	3	No partial reservation; invalid carts and customers rejected
Cancellation and unknown order	2	Resources restored once; unknown order rejected
Cart snapshot	1	Caller mutations cannot change the stored order
Independent subsystems	1	Services work directly and cleanup is idempotent

Run make run for the console demonstration and make test for the checks. The demo buys the only keyboard, cancels that order, rejects Bob's payment, and buys the keyboard again for Alice. The final purchase succeeds only when cancellation has refunded Alice and the failed payment has released the keyboard.

5 Conclusion

OrderFacade hides service creation, sequencing, order ID creation and propagation, placement compensation, and cancellation coordination. Main depends on that small interface and uses Customer and Order as data. This reduces the coordination logic in the client while keeping each service responsible for its own state.

The subsystem classes remain public and have no reference to the facade. Advanced clients can create and call them directly, taking responsibility for coordination and cleanup themselves. The facade therefore provides a convenient entry point without making the underlying subsystem inaccessible.

6 Repository and source basis

https://github.com/adilzhandev/assignment-4-facade-pattern

Implementation reference: commit e55dd96. Source files are under src/main/java/shop; the 12 checks are in src/test/java/shop/OrderFacadeTest.java. The report follows Assignment_4_Facade_Pattern_Instructions.docx, including Option B and the Clean Code requirements. Editable UML source and rendered diagrams are included in docs.